CompSci 471

Artificial Intelligence

Lesson 4: Problem Formulation

Turning real missions → searchable models

A search algorithm solves the problem you represented —not necessarily the problem you intended.

World | **State** | **Actions** | **Goal** | **Cost** | **Search**

Fall 2026

Retrieval warm-up

Lessons 1–3 focused on trust, systems, and inputs.

A system gives a convincing output. → **What version of the problem did the system actually receive?**

Included

What information is included?

Missing

What information is missing?

Irrelevant

What details are irrelevant?

Best

What does “best” mean?

Think-pair-share: name one way a poor representation could make an AI system fail.

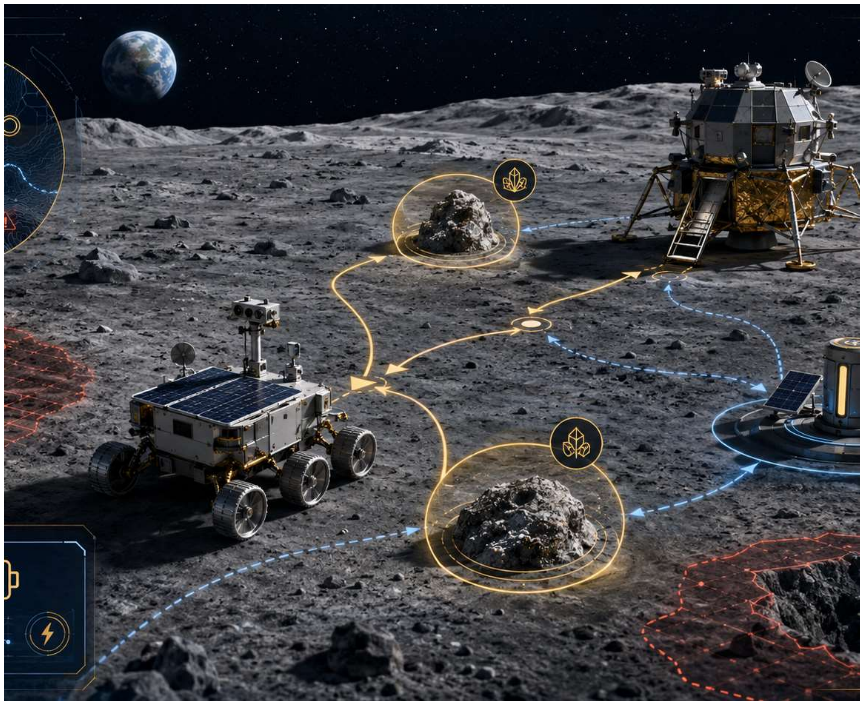
Today’s mission

By the end of Lesson 4, students should be able to…

1. **Define states, actions, goal tests, and path costs**
2. **Formulate real-world problems as search problems**
3. **Explain how problem representation affects solution difficulty**

Running scenario: lunar rover mission

How does a messy mission become a searchable problem?



Mission brief

- Leave the lander
- Collect samples from two sites
- Manage limited battery
- Avoid unstable terrain
- Return to the lander

Question: what must the rover remember to make correct decisions?

From mission to model

Formulation translates a real task into something an algorithm can search.

Mission narrative — natural language task → **Computational representation** — states, actions, goals, costs → **State space** — possible situations → **Search algorithm** — systematic exploration → **Solution path** — sequence of actions

Formulation happens before algorithm selection.

A better algorithm cannot fully rescue a poorly represented problem.

The four core elements

A search problem answers four questions.

State

Where are we?
Decision-relevant situation

Actions

What can we do?
Legal moves from a state

Goal test

When are we done?
Condition that recognizes success

Path cost

What is better?
Cumulative quantity to minimize

Initial state: the specific state where the problem starts.

States: what must the system remember?

A state is a decision-relevant snapshot, not a complete copy of reality.

Too little	Useful abstraction	Too much
location only	(location, samples, battery)	everything measurable
Cannot know whether samples are already collected.	Supports actions, goal recognition, and cost reasoning.	Adds states without improving decisions.

Real world → **State** → Actions → Goal → Cost → Search

Relevant, missing, and irrelevant information

Representation design is an engineering tradeoff.

Relevant

Affects legal actions, goal recognition, or cost

current location
samples collected
battery category

Missing

Needed for correct decisions but absent

sample status
remaining battery
hazard status

Irrelevant

Adds complexity without changing decisions

paint color
serial number
operator name

Engineering check: does the state support actions, goal tests, and costs?

Actions and state transitions

Actions are operations available from a state.

Current state

(site A, sample A collected, medium battery)

- **Move** — to neighboring site
- **Collect** — only at sample site
- **Recharge** — only at station

Successor state

updated location, samples, and battery

Preconditions matter: not every action is legal in every state.

Real world › State › **Actions** › Goal › Cost › Search

Goal tests: when is the mission complete?

The goal test evaluates a state, not the amount of effort spent.

Weak	Incomplete	Complete
Rover is at the lander	**At least one sample collected**	**Both samples collected and rover at lander**
May return before collecting samples	May never return safely	Captures the actual mission objective

A good goal test is precise, checkable, and aligned with the actual objective.

Real world › State › Actions › **Goal** › Cost › Search

Path costs: what does “best” mean?

Changing the cost function can change the preferred solution.

Shortest distance

Minimize meters traveled

Least energy

Minimize battery consumed

Lowest risk

Avoid unstable terrain

Fastest time

Minimize mission duration

Same map. Same goal. Different cost. Different “best” path.

Real world › State › Actions › Goal › **Cost** › Search

Complete rover formulation

Now assemble the pieces into one coherent search problem.

State	(location, samples collected, battery category)
Actions	move, collect sample, recharge, return
Goal test	both samples collected AND rover at lander
Path cost	total energy used, with hazard penalties

This template is reusable across route planning, robotics, cyber response, scheduling, and games.

Representation changes the state space

More expressive representations create more possible states.

Location only

6 locations
6 states

Add samples

6 × 4 sample statuses
24 states

Add battery

6 × 4 × 3 battery categories
72 states

Bigger is not always worse.

The question is whether the added state detail supports correct decisions.

Richer models may be necessary, but unnecessary detail increases search difficulty.

The representation tradeoff

Good abstraction preserves what matters and removes what does not.

Appropriate abstraction

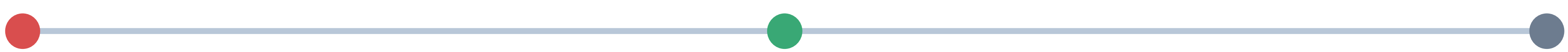
enough information at manageable complexity

Too simple

incorrect or impossible decisions

Too detailed

unnecessary search burden



Use this test: can the state determine legal actions, recognize the goal, and calculate relevant costs?

Debug the formulation

Find the flaw before the algorithm runs.

Inspection vehicle scenario

An underwater vehicle must inspect two bridge supports, avoid restricted zones, manage battery, and return to the dock.

Flawed formulation:

state = current location;
goal = vehicle reaches a support;
cost = number of actions.

Group task

- One missing state feature
- One questionable action or precondition
- One incomplete goal test
- One mismatched path cost

Formulation sprint

Build a search problem from a new mission brief.

Scenario

A medical-delivery robot must move through a damaged building, deliver medicine to two rooms, avoid blocked corridors, and return to the lobby.

State	What must it remember?
Actions	What can it do?
Goal test	When is the mission complete?
Path cost	What should be minimized?

Pairs: create the four-part formulation. Be ready to defend one design choice.

Critique and revise

A formulation is not finished until it survives questions.

Actions

Does the state tell us which actions are legal?

Goal

Can the system recognize success from the state?

Cost

Can the path cost be accumulated over actions?

Complexity

Did we include unnecessary details?

Revision rule: change one element and explain how the change improves the formulation.

Synthesis: problem formulation

Rebuild the search problem template from memory.

State

What information distinguishes situations?

Actions

What can the agent do?

Goal test

How do we know we are done?

Path cost

What makes one solution better?

Good search begins before the search algorithm runs.

Representation shapes what the system can even consider.

Exit Ticket

Apply the formulation template to a compact scenario.

Scenario

A cyber-defense agent must investigate hosts in a network, identify a compromised machine, and minimize disruption to mission-critical services.

Write

- State
- Actions
- Goal test
- Path cost
- One representation tradeoff

Next class: Uninformed Search — once the problem is defined, how should the algorithm explore it?